

Contraintes OCL

Un aéroport(port,gare) est identifié par trois lettres uniques à chaque aéroport.

```
context LieuTransport
inv:
self.code.size() = 3
and LieuTransport.allInstances()->forAll(l1,l2 | l1 <> l2 implies l1
.code <> l2.code)
and self.code.characters()->forAll(c | 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'.
includes(c))
```

Une compagnie aérienne est identifiée par un code de deux lettres unique à chaque compagnie croisière et ligne de train moins de six caractères uniques.

Chaque vol est assigné à un avion appartenant à la compagnie aérienne qui l'exploite.

```
context Vol
inv:
self.compagnie = self.avion.compagnie
```

```
context Exploitant
inv:
if
self.oclIsKindOf(CompagnieAerienne) then self.code.size() = 3
else
self.code.size() < 6 and self.code.size() >= 1
endif
and
Exploitant.allInstances()->forAll(e1,e2 | e1 <> e2 implies e1.code
<> e2.code) and self.code.characters()->forAll(c | '
ABCDEFGHIJKLMNOPQRSTUVWXYZ'.includes(c))
```

Un même avion(et paquebot) peut être assigné à plusieurs vols, mais deux vols assignés au même avion ne peuvent pas se chevaucher.

```
context Paquebot
inv:
self.itineraires->forAll(i1,i2 | (i1 <> i2 and i1.dateHeureDepart <=
    i2.dateHeureDepart) implies i2.dateHeureDepart>= i1.
    dateHeureArrivee)
```

```
context Avion
inv:
self.vols->forAll(v1,v2 | (v1 <> v2 and v1.dateHeureDepart <= v2.
    dateHeureDepart) implies v2.dateHeureDepart >= v1.
    dateHeureArrivee))
```

L'ID complet est unique à chaque vol id : code à deux lettres de la compagnie suivi de quatre chiffres.

```
context Vol
inv:
self.code.size() = 6
and self.code.substring(1,2) = self.compagnie.code
and self.code.substring(3,6).characters()->forAll(c | '0123456789'.
    includes(c))

and Vol.allInstances()->forAll(v1,v2| v1 <> v2 implies v1.code <> v2
    .code)
```

Une même section ne peut apparaître qu'une seule fois, et tout avion doit contenir une section Économique (E) .

```
context Avion
inv:
self.sections->forAll(s1,s2 | s1 <> s2 implies s1.type <> s2.type)
    and self.sections->exists(s| s.type = TypeSectionAvion::
    ECONOMIQUE)
```

Un itinéraire ne peut pas durer plus de 14 jours.

```
context Itineraire
inv:
self.getDuree() <= 14
```

Chaque itinéraire visite une suite de ports fixes comprenant au moins trois ports distincts . Le port de départ et d'arrivée doit être le même .

```
context Itineraire
inv:
self.ports->first() = self.ports->last() and self.ports->asSet()->
    size() >= 3
```

Chaque trajet visite une suite de gares fixes comprenant au moins deux gares. Une même gare ne peut apparaître qu'une seule fois dans un trajet .

```
context Trajet
inv:
self.gares->asSet()->size() >= 2 and self.gares->size() = self.gares
    ->asSet()->size()
```